



NATURAL LANGUAGE FOR DATA VISUALIZATION

**MR. GUNTAWIT
MR. RATCHAPHON
MR. CHALANTORN**

**ANURAKBOONYING
PROONGPATTANASKUL
SAWANGWONGCHINSRI**

**A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING**

**FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY**

2023

NATURAL LANGUAGE FOR DATA VISUALIZATION

MR. GUNTAWIT	ANURAKBOONYING
MR. RATCHAPHON	PROONGPATTANASKUL
MR. CHALANTORN	SAWANGWONGCHINSRI

A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2023

Computer Engineering Project
entitled
NATURAL LANGUAGE FOR DATA VISUALIZATION

Mr. Guntawit Anurakboonying
Researcher

Mr. Ratchaphon Proongpattanaskul
Researcher

Mr. Chalantorn Sawangwongchinsri
Researcher

Asst. Prof. Tanasanee Phienthrakul, Ph.D.
Advisor

Thesis
entitled
NATURAL LANGUAGE FOR DATA VISUALIZATION

was submitted to the Faculty of Engineering & International College,
Mahidol University
for the degree of Bachelor of Engineering (Computer Engineering)
on
April 15, 2026

Asst. Prof. Tanasanee Phienthrakul, Ph.D.
Chair Committee

Committee 1, Ph.D. (Computer Engineering)
Committee

Committee 2, Ph.D. (Computer Engineering)
Committee

BIOGRAPHY

NAME	Mr. Guntawit Anurakboonying
DATE OF BIRTH	01 January 2002
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	111 111 1111
E-MAIL	FirstEmail@gmail.com

NAME	Mr. Ratchaphon Proongpattanaskul
DATE OF BIRTH	15 February 2002
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	222 222 2222
E-MAIL	SecondEmail@yahoo.com

BIOGRAPHY

NAME	Mr. Chalantorn Sawangwongchinsri
DATE OF BIRTH	03 March 2002
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	333 333 333
E-MAIL	ThirdEmail@outlook.com

ACKNOWLEDGEMENT

First and foremost, we would like to express our special thanks and sincere of gratitude to our advisor and co-advisors, Asst. Prof. Tanasanee Phienthrakul, who insight, advise and knowledge us a lot in finalizing this project within the limited time frame.

Beside our advisors, we would like to thank to the committees, Asst. Prof. XXX XXXX, Asst. Prof. YYY YYYY, and Asst. Prof. ZZZ ZZZZ, for their insightful comment and encouragement. Also, the questions which encourage us to rethink and research more about some factors which we missed. Therefore, this project would not be completed without comments, questions, and support from our committees.

Finally, we would like to special thanks to our university, Mahidol University International College, and Faculty of Engineering, Mahidol University, which provided us a lot of resources to study, financial means for researching this project.

Mr. Guntawit Anurakboonying

Mr. Ratchaphon Proongpattanaskul

Mr. Chalantorn Sawangwongchinsri

NATURAL LANGUAGE FOR DATA VISUALIZATION

STUDENTS	Mr. Guntawit Anurakboonying ICCI Mr. Ratchaphon Proongpattanaskul ICCI Mr. Chalantorn Sawangwongchinsri ICCI
DEGREE	Bachelor of Engineering (Computer Engineering)
PROJECT ADVISOR	Asst. Prof. Tanasanee Phienthrakul, Ph.D.
DATE OF GRADUATION	April 15, 2026

ABSTRACT

Data visualization is essential for understanding complex datasets, yet creating effective visualizations often requires technical expertise in programming or visualization tools. This project explores the integration of natural language processing (NLP) techniques to simplify the visualization creation process. By enabling users to describe visualizations through plain English text, the system automatically interprets user intent and generates corresponding visual representations. The proposed approach combines large language models (LLMs) with structured visualization frameworks to map textual descriptions into chart specifications. The study evaluates the system's performance in terms of accuracy, response time, and user satisfaction. Experimental results are expected to demonstrate that natural language interfaces can enhance accessibility and efficiency in data exploration, providing an intuitive bridge between human communication and visual analytics.

KEYWORDS : Natural Language Processing (NLP), Data Visualization, Large Language Models (LLM), Text-to-SQL, Web Application

30 Pages

CONTENTS

	Page
BIOGRAPHY	i
ACKNOWLEDGEMENT	iii
ABSTRACT (ENGLISH)	iv
CONTENTS	v
LIST OF TABLES	viii
LIST OF FIGURES	ix
CHAPTER 1 INTRODUCTION	1
1.1 Background	1
1.2 Objective	2
1.3 Scope	2
1.3.1 Natural Language Processing	2
1.3.2 Data Visualization	2
1.3.3 Data Processing and Management	3
1.4 Expected Results	3
1.5 Timeline	4
CHAPTER 2 LITERATURE REVIEW	5
2.1 A Comparison of Data Visualization Libraries for React [2]	5
2.2 Performance comparison of REST vs GraphQL in different web environments - Node.js and Python [3]	5
2.3 Natural Language Interfaces for Tabular Data Querying and Visualization: A Survey [7]	6
2.4 Refining Zero-Shot Text-to-SQL Benchmarks via Prompt Strategies [8]	6
2.5 Text-to-Pipeline: Bridging Natural Language and Data Preparation Pipelines [4]	7
2.6 Mirror: A Natural Language Interface for Data Querying, Summarization, and Visualization[6]	7

2.7	Summary	8
CHAPTER 3	METHODOLOGY	9
3.1	System Design	9
3.1.1	Overall	9
3.1.2	Presentation Layer	10
3.1.3	Application Layer	10
3.1.4	Data Layer	10
3.2	Frontend (Presentation Layer) — React.js + D3.js	11
3.2.1	Purpose	11
3.2.2	Core Responsibilities	11
3.2.3	Technologies	12
3.3	Backend (Application Layer) — FastAPI	12
3.3.1	Purpose and Overview	12
3.3.2	Query Processing Algorithm	12
3.4	Data Layer	16
3.4.1	Purpose and Overview	16
3.4.2	File System Storage Layer	16
3.4.3	Temporary Database Layer	17
3.4.4	Metadata Layer (PostgreSQL)	18
3.4.5	Data Dictionary	18
3.4.6	Data Flow Architecture	20
3.4.7	Advantages and Trade-offs	20
3.5	Performance Evaluation	21
3.5.1	Natural Language to SQL Translation Evaluation	21
3.5.2	User Experience Evaluation	24
3.5.3	Integration of Evaluation Results	26
CHAPTER 4	RESULT	27
4.1	Results	27
4.2	Discussions	27
CHAPTER 5	Conclusion	28
5.1	Conclusion	28

5.2	Obstacles	28
5.3	Future Work	28
REFERENCES		29

LIST OF TABLES

Table	Page
1.1 Project Timeline	4
3.1 User Table	18
3.2 Dataset Table	19
3.3 DatasetMetadata Table	19
3.4 QueryHistory Table	19

LIST OF FIGURES

Figure	Page
3.1 System Architecture: Natural Language for Data Visualization	10
3.2 Query Processing Algorithm: Natural Language for Data Visualization	13
3.3 Relational Data Model: Natural Language for Data Visualization	18

CHAPTER 1

INTRODUCTION

1.1 Background

Data visualization transforms complex datasets into interpretable visuals that reveal trends and insights. However, traditional tools like Tableau or Power BI require technical expertise, creating barriers for non-technical users.

Natural Language to Visualization (NL2Vis) is a task that transforms natural-language descriptions into visual representations for grounded tables. Unlike SQL queries that specify what data is needed, visualization queries must additionally specify how to visualize it. Early approaches like NL4DV and FlowSense relied on semantic parsers and heuristics, limiting support for complex data transformations.

Recent Large Language Models (LLMs) like GPT-4 have significantly advanced NL2Vis capabilities. Research demonstrates that inference-only models like GPT-3.5 show consistent performance improvements on NL2Vis benchmarks. LLMs excel at understanding natural language context and can interpret user intent to generate appropriate data operations and visualizations. However, effectively leveraging LLMs for NL2Vis requires careful integration with data schemas and preprocessing capabilities.

This work develops a web-based natural language interface that leverages LLMs to enable non-technical users to generate data visualizations without SQL or BI tool expertise. The system allows users to express their analytical needs in plain language—such as "show average sales by region" or "filter customers from Asia and display revenue trends"—and automatically translates these requests into appropriate visualizations.

By combining LLM-powered natural language understanding with web-based accessibility, this system democratizes data exploration and visualization, allowing users to focus on analytical questions rather than technical syntax. The web application interface ensures broad accessibility, making data-driven insights available to business users, analysts, and decision-makers regardless of their technical background.

1.2 Objective

1. To develop a web-based natural language interface that enables non-technical users to generate data visualizations.
2. To enable basic data preprocessing via natural language commands, such as filtering, grouping, sorting, and aggregating data before visualization.

1.3 Scope

1.3.1 Natural Language Processing

1. Translating English natural language queries into structured visualization specifications
 - Support for conversational query patterns (e.g., "show me", "what is", "compare")
 - Intent extraction for visualization requirements (chart type, axes, filters, aggregations)
 - Context-aware query interpretation using LLM (GPT-4 or equivalent)
 - Schema-aware query processing with column and table name recognition
2. Query complexity handling
 - Simple queries: single table, basic filters
 - Medium queries: multi-column analysis, grouping operations
 - Complex queries: multi-step transformations, nested conditions

1.3.2 Data Visualization

1. Supporting basic chart types
 - Bar charts (vertical and horizontal)
 - Line charts (single and multi-series)
 - Pie charts and donut charts
 - Scatter plots
 - Area charts
2. Visualization rendering
 - Interactive charts using D3.js
 - Responsive design for different screen sizes

- Basic chart interactions (hover, tooltips)
- Dynamic chart updates based on query changes

1.3.3 Data Processing and Management

1. Handling basic preprocessing methods
 - Missing value treatment (removal, imputation)
 - Data type normalization and conversion
 - Outlier detection and handling
 - Basic data cleaning operations
2. Data operations via natural language
 - Filtering: row-level conditions, date ranges, value matching
 - Grouping: aggregation by single or multiple columns
 - Sorting: ascending/descending by specified columns
 - Aggregating: sum, average, count, min, max, standard deviation
3. Data storage architecture
 - CSV file storage in file system
 - Temporary database loading using DuckDB for query processing
 - PostgreSQL for metadata storage (user info, dataset catalog, query history)
 - Session-based data caching for performance optimization

1.4 Expected Results

1. Evaluate the accuracy of converting Text-to-SQL using benchmark datasets.
 - Execution Accuracy: Generated query produces correct data output. (70–80% Accuracy)
 - Exact Matching: Output text or JSON spec matches reference exactly. (60–70% Accuracy)
 - Component Matching: Individual parts (chart type, data field, filter, aggregation) are correct. (75–85% Accuracy)
2. User Feedback Evaluation: To assess how effectively the system generates correct visualization types based on natural language input and how easy it is for users to interact with the web interface.

1.5 Timeline

Table 1.1 Project Timeline

Plan	Timeline					
	2025			2026		
	Oct	Nov	Dec	Jan	Feb	Mar
1: Research						
2: Planning						
3: Development						
4: Testing						

CHAPTER 2

LITERATURE REVIEW

This chapter presents a comprehensive review of existing literature and prior work related to AI-assisted data visualization. It outlines key approaches in natural language processing (NLP) and large language models (LLMs) for generating visualizations from textual input. The chapter also discusses methods for evaluating the accuracy, usability, and accessibility of visualization tools, highlighting benchmarks, metrics, and user-centered evaluation strategies. Furthermore, it reviews studies on web-based visualization platforms and prototype applications, providing insights into developing an accessible web application that allows non-expert users to generate meaningful visualizations.

2.1 A Comparison of Data Visualization Libraries for React [2]

This comprehensive comparison examines five major React data visualization libraries: Recharts, Victory, Nivo, react-vis, and Viser, evaluating them based on simplicity, animations, popularity, documentation quality, and integration with D3.js. Capital One The paper specifically discusses how these libraries leverage or integrate with D3.js for data visualization capabilities. The study includes practical implementation of stacked area timeseries charts to assess ease of use and performance characteristics of each library, providing developers with guidance on selecting the appropriate visualization library for React applications.

2.2 Performance comparison of REST vs GraphQL in different web environments - Node.js and Python [3]

This bachelor's thesis conducts a comprehensive performance comparison between REST API and GraphQL in Node.js and Python environments, examining both traditional frameworks (Express and Flask) and performance-centric frameworks (Fas-

tify and FastAPI). The study evaluates performance metrics including response time, throughput, CPU utilization, and memory consumption using JMeter as the testing tool. Results demonstrate that GraphQL outperforms REST in most scenarios, with FastAPI showing 45.81% decrease in response time when using REST and 38% faster performance with GraphQL compared to Flask. The research concludes that FastAPI is particularly suitable for applications requiring high performance when fetching large amounts of data, though it consumes more CPU resources compared to traditional frameworks.

2.3 Natural Language Interfaces for Tabular Data Querying and Visualization: A Survey [7]

This comprehensive survey presents an overview of natural language interfaces for tabular data querying and visualization, enabling users to interact with data using natural language queries. The paper emphasizes semantic parsing as the key technology facilitating translation from natural language to SQL queries or data visualization commands. It delves into recent advancements in Text-to-SQL and Text-to-Vis problems from perspectives of datasets, methodologies, metrics, and system designs, with particular focus on the influence of large language models (LLMs) such as ChatGPT, highlighting their strengths, limitations, and potential for future improvements. Capital OneMedium The survey addresses the critical challenge that traditional methods of interaction with structured data, such as querying with structured query languages or manual plotting, often require significant technical expertise, thereby limiting accessibility to wider user bases. It demonstrates how natural language processing technologies are revolutionizing the way users interact with tabular data, enabling a shift from traditional query languages to more intuitive, language-based interfaces.

2.4 Refining Zero-Shot Text-to-SQL Benchmarks via Prompt Strategies [8]

Focus on improving zero-shot Text-to-SQL systems that use large language models (LLMs) through refined benchmarks and better prompt strategies.

They analyze weaknesses in existing datasets such as logical inconsistencies

and inefficient SQL structures, and propose a cleaned-up benchmark for fairer evaluation.

In addition, they test various prompt strategies to reduce common LLM errors like logic and schema linking mistakes.

Their study contributes improved dataset reliability and provides useful prompt design guidelines that enhance Text-to-SQL accuracy in large language models.

2.5 Text-to-Pipeline: Bridging Natural Language and Data Preparation Pipelines [4]

Text-to-Pipeline (2025) introduces a task that converts natural language (NL) instructions into executable data preparation (DP) pipelines, reducing the need for coding in data preprocessing.

To support this, the authors present PARROT, a benchmark of 18,000 pipelines over 23,009 real-world tables covering 16 DP operations. Current large language models reach only 72.86 accuracy, struggling with complex, multi-step instructions.

Their proposed Pipeline-Agent improves results to 76.17, establishing a key benchmark and baseline for future research in NL-driven data preprocessing automation.

2.6 Mirror: A Natural Language Interface for Data Querying, Summarization, and Visualization[6]

Presents a natural language interface that translates user queries into SQL, then summarizes and visualizes the results.

A key feature is its human-in-the-loop design, allowing users to review and edit generated SQL. The system leverages prompt engineering and schema metadata to improve accuracy.

This work closely aligns with our research goal of enabling non-technical users to query and explore data through natural language, with the added focus in our case on automated preprocessing and visualization.

2.7 Summary

Recent visualization research increasingly combines React.js, D3.js, FastAPI, and Large Language Models (LLMs) to enable seamless and intelligent natural language visualization (NL2Vis) systems. This integration bridges human intent, computational logic, and visual representation, making data visualization more accessible to non-technical users.

D3.js complements React by managing the data-driven rendering layer, translating structured data into expressive and dynamic graphical forms. It enables precise control over visual encodings, scales, and transitions, ensuring that system-generated visualizations remain both accurate and visually coherent.

FastAPI functions as the backend middleware, handling data communication and coordinating interactions between the frontend and the language model. Its asynchronous architecture and high performance make it well suited for real-time visualization workflows that depend on fast response times and continuous model inference.

At the semantic core, LLMs interpret user queries, converting natural language descriptions into structured visualization specifications such as Vega-Lite or JSON schemas. This capability eliminates the need for manual coding, allowing users to create complex visualizations through conversational interaction.

Together, these technologies form an end-to-end intelligent visualization framework in which React.js and D3.js manage interactivity and rendering, FastAPI provides scalable data communication, and LLMs enable intuitive, language-based visualization authoring. This combination establishes a robust foundation for future NL2Vis systems that are both user-friendly and computationally efficient.

CHAPTER 3

METHODOLOGY

This chapter presents the methodology adopted for developing the Natural Language to Data Visualization (NL2Vis) web application. The chapter outlines the overall system architecture. It begins by describing the design approach and technologies employed, including React.js for the user interface, D3.js for visualization rendering, FastAPI for backend communication, and a Large Language Model (LLM) for natural language understanding and visualization generation. Subsequently, the database, system workflow, and integration steps are discussed in detail. Finally, the evaluation methodology is explained, focusing on both system performance and user experience.

3.1 System Design

3.1.1 Overall

This architecture follows a classic three-layer design. The Presentation Layer consists of a client application built with React.js for UI and D3.js for data visualizations, which communicates over the internet with the backend. The Application Layer uses FastAPI as the main server that handles incoming requests, orchestrates logic, and triggers LLM processing when needed. The Data Layer stores and retrieves data through PostgreSQL for structured storage and CSV files for data. Together, these layers form a pipeline where the client sends requests to FastAPI, FastAPI processes logic and LLM tasks, and then interacts with PostgreSQL or CSV files before returning results back to the client.

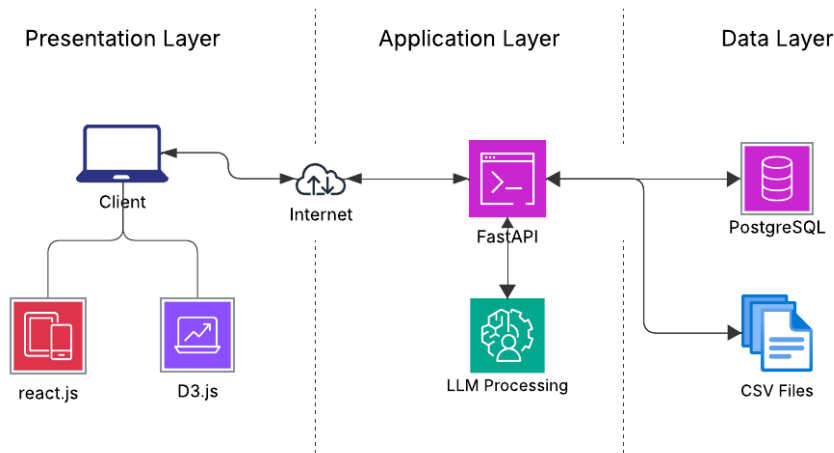


Figure 3.1 System Architecture: Natural Language for Data Visualization

3.1.2 Presentation Layer

The Presentation Layer provides the user-facing interface of the Natural Language for Data Visualization system. It is implemented using React.js for the front-end logic and D3.js for rendering interactive visualizations. Users submit natural language queries through the client interface, and the resulting charts or graphs are displayed directly in the browser. Communication between the client and the backend server occurs over the internet via HTTP requests.

3.1.3 Application Layer

The Application Layer contains the core processing logic of the system. A FastAPI backend handles incoming requests from the client, coordinates data processing, and interacts with the language model (LLM) module. The LLM Processing component interprets user queries, extracts visualization intent, and generates structured instructions for data retrieval and visualization. FastAPI also manages communication between the user interface and the database.

3.1.4 Data Layer

The Data Layer manages all data storage and retrieval operations. PostgreSQL serves as the primary database for storing structured data, metadata, and processed in-

formation. Uploaded CSV files are stored and accessed through this layer, enabling the system to retrieve the data needed to generate visualizations. The Application Layer interacts with PostgreSQL to execute queries and obtain datasets based on user requests.

3.2 Frontend (Presentation Layer) — React.js + D3.js

3.2.1 Purpose

The frontend provides the main user interface for the system. Its purpose is to allow users to type natural language queries, send them to the backend for processing, and view the generated visualizations in a clear and interactive way. This layer is designed to make data exploration simple and accessible for users who may not have technical knowledge. It ensures that all operations, such as query submission, data selection, and chart rendering, happen smoothly and in real time within a web browser.

3.2.2 Core Responsibilities

The frontend manages all interactions between the user and the system. It provides an input area for users to enter queries, a section to select or upload datasets, and a display area for visual results. When a user submits a query, the frontend sends the request to the backend through an API call and shows a loading message while waiting for a response. Once the backend returns a visualization specification, the frontend checks whether the response is valid and then renders the chart on the screen using D3.js.

The visualization process begins when D3.js receives the dataset and chart information from the backend. It uses this data to draw and update visual elements such as bars, lines, or slices depending on the chart type. Charts automatically update when new data is received, allowing users to see changes immediately. The frontend also supports simple user interactions such as hovering over chart elements to see detailed values or switching between different chart types.

Error handling is another important responsibility of the frontend. If a network issue or an invalid backend response occurs, the system displays a short, clear message to inform the user. This prevents confusion and helps users fix the issue or try again. Overall, the frontend acts as the communication bridge between the user and the system, making the entire process of generating visualizations straightforward and reliable.

3.2.3 Technologies

The frontend is developed using **React.js** and **D3.js**. React.js manages the structure and logic of the user interface, dividing the application into smaller reusable components such as the input box, dataset selector, and chart area. This approach keeps the code organized and efficient. D3.js handles all visualization tasks by converting numeric data into graphical forms such as bars, lines, or pie slices.

For communication, the frontend uses **Axios** to send and receive JSON data from the **FastAPI** backend. The user interface is styled using **Tailwind CSS**, which ensures that the layout is clean, consistent, and responsive on different devices. Together, these tools make the frontend efficient, interactive, and easy to use for creating data visualizations through natural language input.

3.3 Backend (Application Layer) — FastAPI

3.3.1 Purpose and Overview

The backend layer serves as the orchestration center of the system, coordinating a two-stage pipeline that transforms natural language queries into data-driven visualizations. It manages the complete workflow from receiving user input through query execution to visualization recommendation, ensuring efficient communication between the frontend interface, the AI processing layer, and the data storage layer.

3.3.2 Query Processing Algorithm

The backend implements a structured four-phase algorithm for converting natural language queries into visualizations:

Phase 1: Natural Language to SQL Translation When a natural language query is received from the frontend, the backend performs the following operations:

1. Query Reception and Validation

- Receive natural language query and dataset identifier from frontend
- Validate user authentication and dataset access permissions
- Retrieve dataset schema from PostgreSQL metadata layer

2. Schema Context Preparation

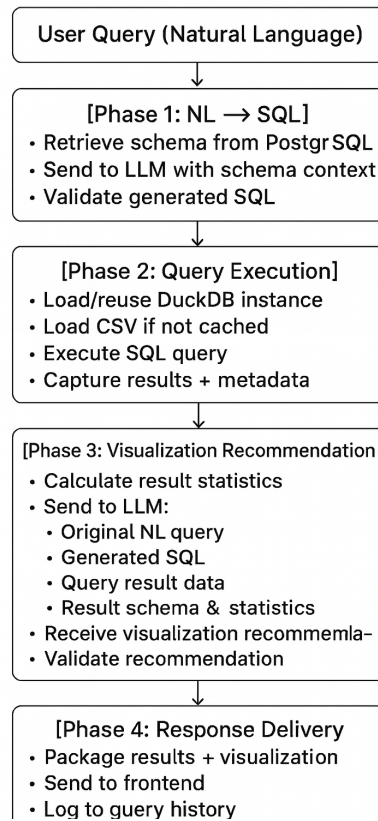


Figure 3.2 Query Processing Algorithm: Natural Language for Data Visualization

- Extract column names, data types, and sample values from metadata
- Construct schema description for LLM context
- Include relevant constraints and relationships

3. LLM-Based SQL Generation

- Send natural language query along with schema context to LLM
- Use specialized prompts engineered for Text-to-SQL translation
- Request structured SQL output with syntax validation
- Receive generated SQL query from LLM

4. SQL Validation and Safety Check

- Parse generated SQL to verify syntax correctness
- Validate that query only performs SELECT operations (no modifications)
- Check for potentially dangerous operations (e.g., excessive joins)
- Sanitize query to prevent SQL injection

Phase 2: Temporary Database Loading and Query Execution The backend manages ephemeral database instances for efficient query processing:

1. Session-Based Database Management

- Check if DuckDB instance already exists for current user session
- If not present, instantiate new in-memory DuckDB instance
- Load CSV file from file system into temporary database
- Cache database instance for session duration

2. Query Execution

- Execute validated SQL query against DuckDB instance
- Set execution timeout (default: 30 seconds) to prevent resource exhaustion
- Capture query results and execution metadata
- Handle execution errors with informative error messages

3. Result Processing

- Convert query results to JSON-serializable format
- Calculate result statistics (row count, column types, value ranges)
- Prepare result summary for visualization recommendation phase
- Limit result size if necessary (e.g., top 1000 rows for large datasets)

Phase 3: Visualization Type Recommendation Rather than allowing the LLM direct access to raw data, the backend provides structured metadata for visualization decisions:

1. Result Metadata Preparation

- Extract result schema (column names and inferred data types)
- Identify relationships between columns based on query structure

2. LLM-Based Visualization Suggestion

- Send the following context to LLM:
 - Original natural language query
 - Generated SQL query
 - Result schema and summary statistics
 - Available visualization types (bar, line, pie, scatter, area)
- Use specialized prompts for visualization recommendation
- Request structured output specifying:

- Recommended chart type
- X-axis and Y-axis mappings
- Color/grouping dimensions (if applicable)
- Aggregation or transformation requirements

3. Visualization Specification Validation

- Verify that suggested chart type is supported
- Validate that specified columns exist in query results
- Check compatibility between data types and visualization type
- Apply fallback rules if recommendation is invalid

Phase 4: Response Generation and Delivery The backend packages all components for frontend rendering:

1. Response Construction

- Combine query results with visualization specification
- Include execution metadata (query time, row count)
- Add original SQL query for transparency (optional user setting)
- Format response as JSON structure

2. Response Delivery

- Send complete package to frontend via HTTP response
- Log query and visualization for history tracking
- Update session cache if needed
- Return appropriate HTTP status codes for error conditions

3.3.2.1 Core Responsibilities

Query Orchestration The backend coordinates the entire query-to-visualization pipeline, managing state transitions between phases and ensuring data consistency throughout the process.

LLM Integration Management Two distinct LLM interactions are managed with different prompt strategies:

- **SQL Generation Prompts:** Emphasize syntactic correctness, schema adherence,

and query safety

- **Visualization Recommendation Prompts:** Focus on data characteristics, user intent, and visualization best practices

Resource Management The backend optimizes resource utilization through:

- Session-based database caching to avoid redundant CSV loading
- Automatic cleanup of expired DuckDB instances
- Query result size limiting to prevent memory overflow
- Timeout enforcement for long-running operations

Error Handling and Recovery Comprehensive error handling at each phase:

- Invalid SQL generation: Return error with explanation, suggest query rephrasing
- Query execution failure: Provide database error details, suggest corrections
- Visualization recommendation failure: Apply rule-based fallback logic
- Timeout errors: Notify user and suggest query simplification

3.4 Data Layer

3.4.1 Purpose and Overview

The storage architecture implements a hybrid model that separates persistent data storage from ephemeral processing capabilities. User-uploaded CSV datasets are maintained permanently in the file system, serving as the authoritative source of truth. When users initiate visualization or analytical queries through natural language, relevant datasets are temporarily loaded into an in-memory database instance, enabling efficient query processing without the overhead and cost of permanent database maintenance. This architectural decision balances data durability, storage economics, and query performance.

3.4.2 File System Storage Layer

3.4.2.1 Purpose and Overview

The file system layer serves as the permanent storage foundation, maintaining all user-uploaded CSV datasets in an organized directory structure with accompanying

metadata files for rapid discovery.

3.4.2.2 Core Responsibilities

The file system layer organizes CSV files in hierarchical directories with each user allocated a dedicated directory for namespace isolation. Metadata files stored alongside each CSV contain dataset identifiers, upload timestamps, file sizes, row counts, and column schemas with inferred data types. The layer implements CRUD operations with validation, security safeguards, versioned updates, and soft deletion mechanisms.

3.4.2.3 Technology Stack

Standard POSIX-compliant file systems provide the storage foundation with operating system-level access controls and backup integration. Metadata is stored in JSON format for portability and human readability.

3.4.3 Temporary Database Layer

3.4.3.1 Purpose and Overview

The temporary database layer provides ephemeral SQL query capabilities, instantiated on-demand when users initiate data visualization or analytical operations. This layer is designed to be lightweight, fast-loading, and automatically cleaned up after query completion.

3.4.3.2 Core Responsibilities

The temporary database layer creates in-memory database instances for each user session, loads CSV files using optimized bulk import operations, executes analytical SQL queries generated from natural language inputs, and releases resources upon query completion or session timeout. Each instance is isolated to prevent cross-user data access.

3.4.3.3 Technology Stack

DuckDB was selected as the temporary database engine for its in-process architecture, vectorized execution optimized for analytical workloads, zero-configuration de-

ployment, native CSV reading with automatic type inference, and intelligent memory management with disk spillover capabilities [?].

3.4.4 Metadata Layer (PostgreSQL)

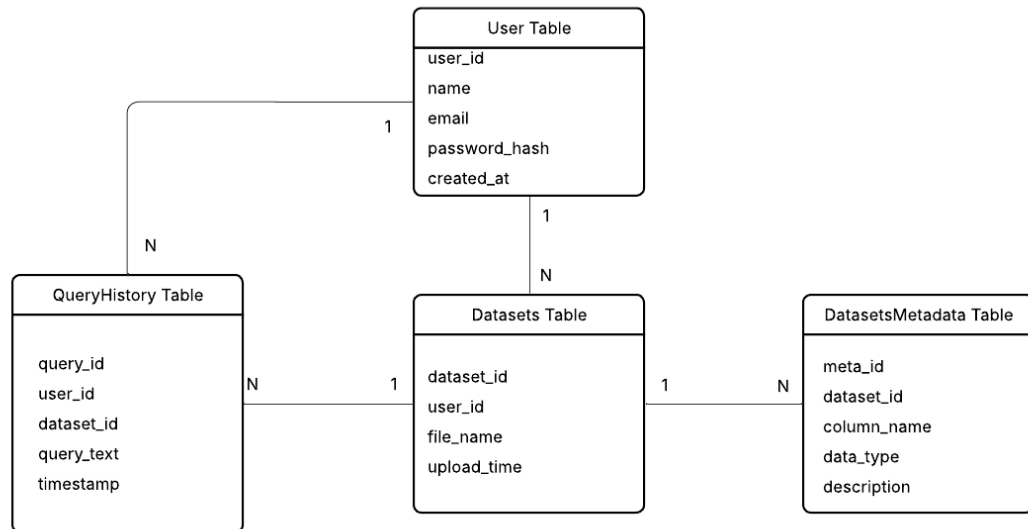


Figure 3.3 Relational Data Model: Natural Language for Data Visualization

3.4.5 Data Dictionary

This section describes the data dictionary for the system, including all core tables used to store user information, datasets, metadata, and visualization settings. Each subsection provides a detailed explanation of the table structure and purpose.

3.4.5.1 User Table

Table 3.1 User Table

Field	Type	Description
user_id	INT (PK)	Unique identifier for each user.
name	VARCHAR	Full name of the user.
email	VARCHAR	User email used for login.
password_hash	VARCHAR	Secure password hash.
created_at	TIMESTAMP	Account creation timestamp.

3.4.5.2 Dataset Table

Table 3.2 Dataset Table

Field	Type	Description
dataset_id	INT (PK)	Unique identifier for each dataset.
user_id	INT (FK)	Owner of the dataset.
file_name	VARCHAR	Original uploaded file name.
uploaded_time	TIMESTAMP	Time of dataset upload.

3.4.5.3 DatasetMetadata Table

Table 3.3 DatasetMetadata Table

Field	Type	Description
meta_id	INT (PK)	Unique metadata entry ID.
dataset_id	INT (FK)	Dataset this metadata belongs to.
column_name	VARCHAR	Column name in the dataset.
data_type	VARCHAR	Inferred data type (numeric, string, date).
description	TEXT	Auto-generated or user-provided description.

3.4.5.4 QueryHistory Table

Table 3.4 QueryHistory Table

Field	Type	Description
query_id	INT (PK)	Unique identifier for each recorded query.
user_id	INT (FK → User.user_id)	User who made the query.
dataset_id	INT (FK → Datasets.dataset_id)	Dataset used for the query.
query_text	TEXT	The natural language query entered by the user.
timestamp	DATETIME	Time when the query was executed.

3.4.5.5 Purpose and Overview

The metadata layer maintains a lightweight PostgreSQL database storing user accounts and dataset catalog information. PostgreSQL stores zero bytes of actual CSV

data, functioning solely as a metadata index enabling the LLM to discover available datasets and their schemas before loading CSV files into temporary DuckDB instances.

3.4.5.6 Core Responsibilities

The metadata layer stores user authentication credentials with bcrypt-hashed passwords, maintains a dataset registry with file system references and column schemas in JSONB format for LLM query planning, records query history for system analytics without storing actual results, and tracks active sessions to enable intelligent caching and resource cleanup.

3.4.5.7 Technology Stack

PostgreSQL (version 15.x) was selected for its ACID compliance, concurrent access support through MVCC, and native JSONB indexing for efficient schema queries [5]. SQLAlchemy (version 2.0.x) provides the ORM layer with declarative models, async support, and connection pooling [1]. The metadata database typically consumes less than 500MB even with thousands of users and datasets.

3.4.6 Data Flow Architecture

The complete data flow proceeds as follows: users upload CSV files through the web interface, the backend validates and writes files to the file system while extracting metadata to PostgreSQL, users submit natural language queries, the LLM queries PostgreSQL to identify target datasets and retrieve schemas, the orchestration layer checks for cached databases or instantiates new DuckDB instances and loads CSVs, generated SQL queries execute against the temporary database, results return to the visualization module, and temporary databases persist for the session duration before cleanup.

3.4.7 Advantages and Trade-offs

The hybrid architecture provides significant storage cost reduction compared to permanent database storage, simplified backup through standard file system tools, data portability with easily exportable CSV files, superior query performance through columnar storage, resource efficiency with on-demand allocation, and horizontal scalability.

Trade-offs include initial loading latency for first queries on a dataset, mitigated through session caching, and complexity in managing the coordination between storage layers.

3.5 Performance Evaluation

This chapter presents a comprehensive evaluation of the system through two complementary assessment methodologies. The evaluation framework is designed to validate both the technical accuracy of the natural language processing capabilities and the practical usability of the web application interface. Section 3.5.1 describes the quantitative assessment of the natural language to SQL translation module using the BIRD benchmark dataset, while Section 3.5.2 presents the qualitative evaluation of visualization generation and user experience through structured user surveys.

3.5.1 Natural Language to SQL Translation Evaluation

3.5.1.1 Evaluation Objectives

The primary objective of this evaluation is to assess the accuracy and robustness of the natural language to SQL (NL2SQL) translation component, which forms the foundation of the system's query processing pipeline. Specifically, this evaluation aims to:

- Measure the system's ability to correctly interpret natural language queries and generate syntactically valid SQL statements
- Evaluate the semantic accuracy of generated queries against ground truth SQL
- Identify common failure patterns and limitations in query understanding
- Compare the system's performance against established baselines in the NL2SQL domain

3.5.1.2 BIRD Benchmark Dataset

The BIRD (BIg Bench for LaRge-scale Database Grounded Text-to-SQL Evaluation) benchmark dataset was selected as the evaluation framework for assessing NL2SQL translation performance. BIRD represents a state-of-the-art benchmark specifically designed to evaluate text-to-SQL systems in realistic, large-scale database scenarios. The dataset encompasses several characteristics that make it particularly suitable for this evaluation:

Dataset Characteristics

- **Scale and Diversity:** BIRD contains over 12,000 unique question-SQL pairs spanning 95 databases across multiple domains, including finance, healthcare, education, and e-commerce. This diversity ensures comprehensive evaluation across various data schemas and query complexities.
- **Realistic Complexity:** Unlike synthetic benchmarks, BIRD includes queries that reflect real-world analytical tasks, incorporating complex SQL constructs such as nested subqueries, multi-table joins, aggregations, and window functions.
- **Database Scale:** The benchmark utilizes databases with substantial table counts (average of 8.4 tables per database) and realistic data volumes, testing the system's ability to handle enterprise-scale data environments.
- **Question Difficulty Levels:** BIRD categorizes questions into four difficulty tiers (easy, medium, hard, extra hard) based on SQL complexity, number of tables involved, and reasoning requirements, enabling granular performance analysis.

3.5.1.3 Evaluation Metrics

The NL2SQL component is evaluated using standard metrics established in the text-to-SQL research community:

Execution Accuracy (EX) Execution accuracy measures whether the SQL query generated by the system produces the same result set as the ground truth SQL query when executed against the database. This metric is calculated as:

$$EX = \frac{\text{Number of queries with matching results}}{\text{Total number of test queries}} \times 100\% \quad (3.1)$$

Execution accuracy serves as the primary metric as it directly measures functional correctness from an end-user perspective, regardless of syntactic variations in the generated SQL.

Valid Efficiency Score (VES) The Valid Efficiency Score evaluates whether generated SQL queries are not only correct but also efficiently structured. A query receives full

VES credit if it produces correct results without unnecessary computational overhead (e.g., avoiding redundant joins or subqueries). This metric is particularly relevant for production systems where query performance impacts user experience.

Error Analysis Categories Failed queries are categorized into the following error types for detailed analysis:

- **Schema Understanding Errors:** Incorrect table or column selection
- **Join Logic Errors:** Improper relationship mapping between tables
- **Aggregation Errors:** Incorrect use of GROUP BY, HAVING, or aggregate functions
- **Filtering Errors:** Misinterpretation of WHERE clause conditions
- **Syntax Errors:** Generated SQL fails to parse
- **Semantic Errors:** Query executes but produces incorrect results

3.5.1.4 Experimental Setup

Evaluation Protocol The evaluation follows a systematic protocol:

1. Each natural language question from the BIRD test set is input to the system
2. The system generates SQL queries using the NL2SQL translation module
3. Generated queries are executed against the corresponding BIRD database
4. Results are compared against ground truth query results
5. Execution time and token usage (for LLM-based approaches) are recorded
6. Failed queries are manually analyzed and categorized by error type

To ensure fair comparison, the evaluation excludes training examples if few-shot learning is employed, and maintains consistent prompting strategies across all test cases.

3.5.1.5 Baseline Comparisons

The performance of the proposed system is compared against established baselines:

- **BIRD benchmark leaders:** Published results from top-performing systems on the BIRD leaderboard
- **General-purpose LLMs:** Direct prompting of models like GPT-4 or Claude

without specialized fine-tuning

This comparative analysis contextualizes the system's performance within the current state-of-the-art in NL2SQL translation.

3.5.2 User Experience Evaluation

3.5.2.1 Evaluation Objectives

The second component of the performance evaluation focuses on assessing the practical usability and effectiveness of the web application from an end-user perspective. This evaluation aims to:

- Evaluate the appropriateness and accuracy of automatically generated visualizations
- Assess the overall user experience and interface usability
- Identify usability issues and areas for improvement
- Measure user satisfaction and perceived usefulness of the system
- Validate the system's ability to serve non-technical users effectively

3.5.2.2 Survey Design and Methodology

Participant Recruitment A diverse participant pool was recruited to ensure comprehensive evaluation.

- **Target Sample Size:** 10-20 participants
- **Recruitment Method:** Convenience sampling through university networks and professional communities

Survey Instrument The evaluation employs a mixed-methods survey instrument combining quantitative ratings and qualitative feedback:

Quantitative Assessment (5-point Likert Scale)

- **Visualization Quality:**
 - Appropriateness of automatically selected chart types
 - Clarity and readability of generated visualizations
 - Accuracy of data representation
 - Aesthetic quality of visual design

- **System Usability:**
 - Ease of formulating natural language queries
 - Speed and responsiveness of the system
 - Clarity of error messages and feedback
 - Overall interface intuitiveness
- **User Satisfaction:**
 - Likelihood to use the system for real tasks
 - Preference compared to traditional BI tools
 - Confidence in generated insights
 - Overall satisfaction with the experience

Qualitative Feedback

Open-ended questions capture nuanced user perspectives:

- What aspects of the system did you find most useful?
- What difficulties did you encounter while using the system?
- Were there instances where the visualization did not meet your expectations?
Please describe.
- What improvements would you suggest?
- In what scenarios would you use this system in your work or studies?

3.5.2.3 Evaluation Metrics

Visualization Appropriateness Score This metric assesses whether the automatically generated visualization type is suitable for the data and query intent. Expert evaluators rate each generated visualization on a scale of 1--5:

- **5:** Optimal visualization choice, effectively communicates the insight
- **4:** Good choice, minor improvements possible
- **3:** Acceptable but suboptimal visualization
- **2:** Poor choice, significantly hinders understanding
- **1:** Completely inappropriate or misleading visualization

The overall Visualization Appropriateness Score is calculated as the mean rating across all evaluated query-visualization pairs.

System Usability Scale (SUS) The standardized System Usability Scale (SUS) questionnaire is administered to provide a validated measure of perceived usability [?]. The SUS consists of 10 items rated on a 5-point scale, yielding a composite score from 0-100. Scores above 68 are considered above average usability, while scores above 80 indicate excellent usability.

3.5.3 Integration of Evaluation Results

The findings from both evaluation components are synthesized to provide a holistic assessment of the proposed system. The NL2SQL evaluation validates the technical foundation, while the user survey confirms practical utility. Discrepancies between technical accuracy and perceived usability inform targeted improvements, ensuring the system achieves both computational correctness and user-centered design objectives.

CHAPTER 4

RESULT

This chapter reports your experiment results or study report proposed in previous chapter. It typically consists of 1) Results in terms of tables or figures and 2) explanation or discussion of your results, study, or key findings.

4.1 Results

(Experiment/Results in the forms of tables or/and figures)

4.2 Discussions

(Interpretation or the meaning of your experiment/results)

CHAPTER 5

CONCLUSION

This chapter summarizes all of your work. It typically starts with the brief explanation of what you do such as the objective of your work, design, data, experiment results, key findings, interpretation of your experiment and/or result. The obstacles of your work can also be discussed and finally followed by future work.

5.1 Conclusion

...

5.2 Obstacles

...

5.3 Future Work

...

REFERENCES

- [1] M. Bayer. Sqlalchemy. In A. Brown and G. Wilson, editors, *The Architecture of Open Source Applications Volume II: Structure, Scale, and a Few More Fearless Hacks*. aosabook.org, 2012. URL <http://aosabook.org/en/sqlalchemy.html>.
- [2] Capital One Tech. A comparison of data visualization libraries for react. 2024. URL <https://www.capitalone.com/tech/software-engineering/comparison-data-visualization-libraries-for-react/>. Software Engineering & Web Development.
- [3] D. Demir and E. Nilsson. Performance comparison of rest vs graphql in different web environments - node.js and python, May 2023. Computer Science.
- [4] Y. Ge, Y. Liu, Y. Mao, and Y. Gao. Text-to-pipeline: Bridging natural language and data preparation pipelines. *arXiv preprint arXiv:2505.15874*, May 2025. doi: 10.48550/arXiv.2505.15874. Applied Science & Technology Source Ultimate.
- [5] M. Stonebraker and L. A. Rowe. The design of postgres. *ACM SIGMOD Record*, 15(2):340--355, 1986. doi: 10.1145/16856.16888.
- [6] C. Xu, J. McAuley, and P. Wang. Mirror: A natural language interface for data querying, summarization, and visualization. Mar. 2023. doi: 10.1145/3543873.3587309. arXiv preprint arXiv:2303.08697; Applied Science & Technology Source Ultimate.
- [7] W. Zhang, Y. Shen, Y. Huang, Y. Wang, Z. Li, Y. R. Chen, D. Jiang, W. Wang, and J. Han. Natural language interfaces for tabular data querying and visualization: A survey. *arXiv preprint arXiv:2310.17894*, May 2024. Natural Language Processing & Data Visualization.
- [8] R. Zhou and F. Zhang. Refining zero-shot text-to-sql benchmarks via prompt strate-

gies with large language models. *Applied Sciences*, 15(10), May 2025. ISSN 2076-3417. Applied Science & Technology Source Ultimate.